

# Skynet 第一课：从启动命令到 service/main.lua

把一次真实的 Skynet 启动过程完整走一遍

项目：simbiwu/skynet-mmo-learning

工程基线：Skynet v1.8.0 / Skynet Bundled Modified Lua 5.4.7 / Linux 或 Windows + WSL2

仓库基线 Commit：b6a55b59369565dbe7fd82237f1d1020aaff543c

适合读者：已经做过 C++/Lua MMO Server，希望系统理解 Skynet，而不是只会调用 API

## 阅读导航

这份第一课按一次真实启动的时间顺序组织。第一次阅读建议不要跳着看：

- **第 0~3 节**：先建立全局图，弄清工程如何拉取和编译 Skynet；
- **第 4~7 节**：从启动命令进入 `skynet_main.c` 和 `skynet_start.c`；
- **第 8~12 节**：重点理解 Worker、`snlua`、`loader.lua`、`bootstrap.lua` 和 `newservice`；
- **第 13~16 节**：回到本工程 `service/main.lua`，建立 Service / Thread / Lua State / Coroutine 的完整模型；
- **第 17~21 节**：实际调试、故障注入、自测；
- **第 22 节及附录**：下一课衔接与源码/命令索引。

如果你已经熟悉构建，可以快速浏览第 2~3 节，但第 5、8、9、11、12、14 节不建议跳过。

---

## 0. 这一课到底要解决什么

这一课只做一件事：**把下面这条命令背后发生的事情真正弄清楚。**

```
./third_party/skynet/skynet config/game.lua
```

如果只看表面，这和运行一个普通 Server 程序没有什么区别：给可执行文件传一个配置文件，然后服务器启动。但 Skynet 的启动过程里同时存在 C Runtime、C Service、Lua State、Lua Loader、系统 Service、业务 Service、Worker Thread 和 Coroutine。第一次接触时最容易出现的问题，不是某个 API 不会用，而是这些层混在了一起。

所以本课不按照“文件 A、文件 B、文件 C”罗列源码，而是严格按照**时间顺序**往下走：

1. 工程是怎么把 Skynet v1.8.0 拉下来并编译出来的；
2. `config/game.lua` 到底是谁执行的；
3. `thread = 8` 到底创建了什么；
4. `bootstrap = "snlua bootstrap"` 到底是什么意思；
5. `snlua` 为什么会产生一个独立 Lua State；
6. `loader.lua` 怎样找到 `bootstrap.lua`；
7. `bootstrap.lua` 又怎样启动 `main.lua`；
8. 本工程的 `service/main.lua` 为什么按现在的顺序启动各个 Manager；
9. 最后再把 Service / Thread / Lua State / Coroutine 四者的关系重新串起来。

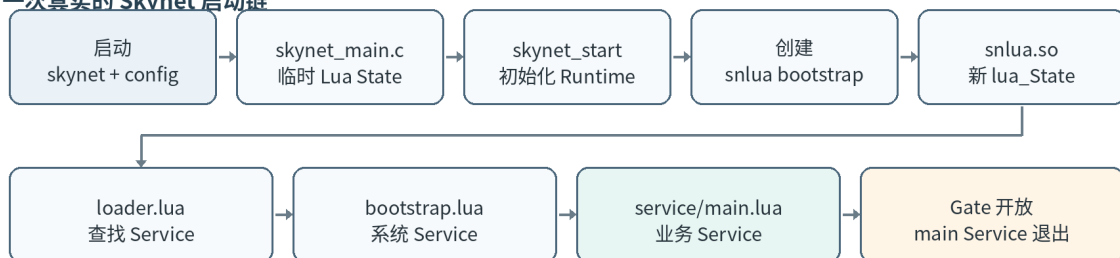
如果这一课学完，你能脱离文档自己画出整个启动链，并且遇到启动错误时知道该去 C Runtime、Loader 还是业务 Service 查，那么这一课就算完成。

### 先记住一个总原则

Skynet 不是“一个 Lua 程序外面包了一层 C”。它更像一个 Actor Runtime：C Runtime 负责调度、消息队列、网络、定时器和 Service 生命周期；大量业务 Service 以独立 Lua State 的形式运行在这个 Runtime 之上。

## 1. 先看全局图，不要一上来钻源码

一次真实的 Skynet 启动链



先把上图记成一句话：

```

启动 skynet 可执行文件
-> 临时 Lua State 读取 config/game.lua
-> C Runtime 初始化
-> 创建 snlua bootstrap
-> snlua 创建自己的 Lua State
-> loader.lua 找到 bootstrap.lua
-> bootstrap.lua 建立 Skynet 系统 Service
-> bootstrap.lua 再启动我们自己的 main.lua
-> main.lua 启动 MMO 业务 Service
-> Gate 最后开放端口

```

这张图里最容易混淆的是三个“Lua”：

- **配置阶段临时 Lua State**：只负责执行 config/game.lua，执行完就销毁；
- **snlua bootstrap 的 Lua State**：属于 bootstrap Service；
- **以后每一个 snlua Service 的 Lua State**：例如 scene\_mgr、player\_mgr、player\_agent，各自拥有自己的 Lua State。

也就是说，Skynet 并不是整个进程只有一个 Lua VM。

如果你熟悉的 C++ MMO Server 来类比：传统框架里可能是一个 GameServer 进程嵌一个 Lua VM，然后所有玩法模块共用它；Skynet 更像 Runtime 不断创建很多“带独立 Lua VM 的 Actor 实例”。这个区别会直接影响后面的状态所有权、配置共享、GC、Service 粒度和内存占用。

## 2. 从工程入口开始：我们到底执行了什么

### 2.1 在当前仓库里，推荐的构建入口

根目录 Makefile 已经把常用命令收敛好了：

```

bootstrap:
    ./scripts/bootstrap_skynet.sh

build:
    ./scripts/linux/build.sh

server:
    ./scripts/linux/run_server.sh

```

所以学习阶段可以直接在仓库根目录执行：

```

make bootstrap
make build
make server

```

你也可以直接运行对应脚本。这里的 Makefile 本身没有做复杂构建逻辑，它更像一个统一入口。

### 2.2 bootstrap\_skynet.sh 做了什么

文件：scripts/bootstrap\_skynet.sh

工程把 Skynet 版本固定为：

```
VERSION="v1.8.0"
DEST="third_party/skynet"
```

核心命令是：

```
git clone --recursive --branch "$VERSION" --depth 1 \
  https://github.com/cloudwu/skynet.git "$DEST"
```

这一步的意义非常简单，但对学习工程非常重要：**所有教程、源码路径和行为都建立在固定版本上。**

如果一边看 v1.8.0 的教程，一边让仓库自动追 Skynet master，那么半年之后 Lua 版本、目录结构、API 行为、源码行号 都可能已经变了。学习工程最忌讳这种漂移。

脚本还有一段针对 WSL 的处理：当仓库位于 /mnt/\* Windows 挂载盘，并且能调用 powershell.exe 时，会转交 Windows PowerShell wrapper 使用原生 Git 拉取源码。这个逻辑主要是规避 DrvFs 权限/chmod 一类问题，与 Skynet Runtime 本身无关。

## 2.3 build.sh 做了什么

文件：scripts/linux/build.sh

```
if [[ ! -f third_party/skynet/Makefile ]]; then
  ./scripts/bootstrap_skynet.sh
fi

make -C third_party/skynet linux
```

第一句保证上游 Skynet 已经存在；第二句才是真正进入 Skynet 上游目录构建 Linux Target。

因此你在 VS Code、Makefile 或 Shell 里看到不同入口时，不需要认为它们是三套构建系统。最终都收敛到：

```
make -C third_party/skynet linux
```

## 3. make linux 之后到底生成了什么

这一部分不要死记文件名。你真正需要分清的是 **四类产物属于四个不同层次。**

### 3.1 第一类：Skynet 主程序

```
third_party/skynet/skynet
```

它是最终启动的 Linux ELF 可执行文件，负责建立整个 Runtime。你后面看到的：

- Handle;
- Service Context;
- Message Queue;
- Worker Scheduler;
- Timer;
- Socket;

- Module Loader;

都属于这个 Runtime 的核心职责。

如果修改的是：

```
third_party/skynet/skynet-src/*.c
```

例如 skynet\_start.c、skynet\_server.c、skynet\_mq.c，就必须重新编译并重新启动整个进程。

## 3.2 第二类：Bundled Lua

典型产物：

```
third_party/skynet/3rd/lua/liblua.a
third_party/skynet/3rd/lua/lua
```

本工程基于 Skynet v1.8.0 自带的改造版 Lua 5.4.7。

这里容易误解的一点是：

```
liblua.a
```

被链接进入 Skynet 主程序，而：

```
third_party/skynet/3rd/lua/lua
```

是一个独立的 Lua 可执行程序，可以拿来跑纯 Lua 单测或学习客户端。

它们来自同一套 Lua 源码，但运行后是两个不同进程，不可能共享 Lua State。

## 3.3 第三类：C Service

典型目录：

```
third_party/skynet/cservice/
```

其中你现在最需要认识的是：

```
snlua.so
logger.so
```

Skynet Runtime 会通过 cpath 找到这些 .so 并动态加载。

这里先重点记住：

**snlua 本身不是一个 Lua 脚本，而是 C Service。**

后面 bootstrap = "snlua bootstrap" 中前面的 snlua，就是这里的 C Service Module。

## 3.4 第四类：Lua C Module

典型目录：

```
third_party/skynet/luaclib/
```

例如：

```
skynet.so
client.so
```

```
sproto.so
lpeg.so
```

它们虽然也是 .so，但加载方式完全不同。

C Service:

```
Skynet Runtime Module Loader
-> cpath
-> snlua.so
```

Lua C Module:

```
Lua require()
-> package.cpath / lua_cpath
-> skynet.so / sproto.so
```

所以以后看到一个 .so，不要简单认为“都是 Skynet 插件”。至少要先判断它属于 C Service 还是 Lua C Module。

### 3.5 现在先不要深挖 jemalloc

原来的资料在这里很早就进入 jemalloc、符号前缀、Profiling 等细节。那些内容是有价值的，但放在第一遍理解启动链时会打断主线。

这一课只需要知道：Skynet v1.8.0 的构建会把 jemalloc 作为 Runtime 内存基础设施的一部分编进去，后续做内存分析、jmem、Profiling 时我们再单独深入。

## 4. 真正启动：run\_server.sh 只是把配置文件交给 Skynet

文件：scripts/linux/run\_server.sh

当前代码非常简单：

```
CONFIG="${1:-config/game.lua}"
exec ./third_party/skynet/skynet "$CONFIG"
```

也就是说：

```
make server
```

最终还是变成：

```
./third_party/skynet/skynet config/game.lua
```

从这一刻起，Shell 脚本已经完成任务，下面开始真正进入 Skynet C Runtime。

## 5. 第一步源码：skynet\_main.c::main

上游文件：third\_party/skynet/skynet-src/skynet\_main.c

这一层最值得你先建立一个非常明确的模型：

config/game.lua  
不是某个业务 Service require 进来的。

它是在 C main() 里临时创建一个 Lua State 来执行的。

## 5.1 argv[1] 就是 config/game.lua

启动：

```
./third_party/skynet/skynet config/game.lua
```

于是：

```
argv[0] = ./third_party/skynet/skynet
argv[1] = config/game.lua
```

main() 首先拿到配置文件路径，然后进行 Skynet 全局环境初始化。

把源码压缩成容易看的伪代码，大致是：

```
int main(int argc, char *argv[]) {
    config_file = argv[1];

    skynet_globalinit();
    skynet_env_init();

    lua_State *L = luaL_newstate();
    luaL_openlibs(L);

    // 用临时 Lua State 执行 config/game.lua
    load_config(config_file);

    // 把结果写进 Skynet Env
    _init_env(L);

    // 配置 Lua State 到这里就没用了
    lua_close(L);

    // 从 Env 读取 thread/cpath/bootstrap 等
    struct skynet_config config = ...;

    skynet_start(&config);
}
```

这里真正重要的不是每个 C API，而是生命周期。

## 5.2 为什么要用 Lua 来写配置

Skynet 没有自己设计一套 INI/YAML Parser，而是直接让配置文件本身就是 Lua 代码。

所以 config/game.lua 可以写：

```
root = "./"
skynet_root = root .. "third_party/skynet/"

luaservice = root .. "service/?.lua;" .. skynet_root .. "service/?.lua"
```

这种表达式会先在临时 Lua State 里计算完，最终得到字符串结果。

但要注意：这个 Lua State 不是任何 Service 的 Lua State。

配置执行完以后，它马上被：

```
lua_close(L);
```

销毁。

所以你不能期待：

```
-- config/game.lua
foo = { a = 1 }
```

然后某个 PlayerAgent 里直接访问到同一张 foo table。

两者根本不是同一个 Lua State，甚至配置 Lua State 在 Service 出现前就已经不存在了。

## 5.3 配置最后存到哪里

skynet\_main.c 中的 \_init\_env() 会遍历配置执行结果，把键值写入 Skynet 的全局环境存储。

因此业务 Service 后面读取配置时用的是：

```
skynet.getenv("gate_port")
```

而不是共享 config/game.lua 当时的 Lua 变量。

这一点可以用你熟悉的方式理解：

```
config/game.lua
  ↓
临时 Configuration Parser
  ↓
Skynet Runtime KV Environment
  ↓
skynet.getenv()
```

不是：

```
config.lua 全局 Table
  ↓
所有 Service 直接共享引用
```

## 5.4 为什么 skynet.getenv() 拿到的经常是字符串

本工程当前配置：

```
thread = 8
gate_port = 8888
max_client = 10000
```

但在 service/main.lua 中又专门有：

```
local function getenv_int(name, default)
    return assert(tonumber(getenv(name, tostring(default))))
end
```

不是多此一举。

因为配置阶段 \_init\_env() 最终把值放进 Runtime Environment，读取回来时按字符串处理。因此业务层通常需要自己做：

```
tonumber(skynet.getenv("gate_port"))
```

这也是为什么配置值适合放简单标量，不适合期待直接保存复杂 Lua Table。

## 6. 用本工程的 config/game.lua 把配置项分层

文件：config/game.lua

当前主要内容可以按用途分成四组。

### 6.1 Service / Lua 搜索路径

```
root = "./"
skynet_root = root .. "third_party/skynet/"

luaservice = root .. "service/?.lua;" .. skynet_root .. "service/?.lua"
lualoader = skynet_root .. "lualib/loader.lua"
lua_path = root .. "lualib/?.lua;" .. root .. "lualib/?/init.lua;" ..
           skynet_root .. "lualib/?.lua;" .. skynet_root .. "lualib/?/init.lua"
lua_cpath = skynet_root .. "luaclib/?.so"
cpath = skynet_root .. "cservice/?.so"
```

这里一定要分清：

| 配置         | 谁使用            | 用来找什么                              |
|------------|----------------|------------------------------------|
| cpath      | Skynet Runtime | C Service，例如 snlua.so              |
| luaservice | loader.lua     | Lua Service 文件，例如 service/main.lua |
| lualoader  | snlua          | 每个 Lua Service 的统一 Loader          |
| lua_path   | Lua require    | 普通 Lua Module                      |
| lua_cpath  | Lua require    | Lua C Module                       |

例如：

```
skynet.newservice("scene/scene")
```

最终是按：

```
luaservice
```

寻找：

```
service/scene/scene.lua
```

而：

```
require "scene.aoi"
```

走的是：

```
lua_path / package.path
```

这是两个完全不同的查找机制。

## 6.2 Runtime 配置

```
thread = 8
harbor = 0
bootstrap = "snlua bootstrap"
start = "main"
logger = nil
```

其中第一课最关键的是：

```
thread = 8
bootstrap = "snlua bootstrap"
start = "main"
```

这三个配置看起来都像“启动参数”，但它们被消费的层级并不相同。

- `thread`: C Runtime 直接读取；
- `bootstrap`: C Runtime 直接读取，并用它创建第一个 Bootstrap Service；
- `start`: C Runtime 自己不会直接创建 `main`，它只是把这个值留在环境里，后面由 `service/bootstrap.lua` 读取。

所以：

```
bootstrap != start
```

而且这不是命名差异，是两个不同阶段。

## 6.3 本项目业务配置

```
gate_host = "0.0.0.0"
gate_port = 8888
max_client = 10000
debug_console_port = 8000

storage_driver = "memory"
storage_pool = 4
```

Skynet C Runtime 并不知道 `gate_port` 是什么，也不知道 `storage_driver` 是什么。

这些只是被放进 Runtime Env，后面由我们自己的：

```
service/main.lua
service/storage/*
service/gateway/*
```

主动读取。

这和很多自研 C++ 框架里的 Configuration Center 很像：底层只负责保存 KV，具体键的业务含义由上层模块解释。

## 7. 第二步源码：skynet\_start(&config) 建立 Runtime

上游文件：third\_party/skynet/skynet-src/skynet\_start.c

skynet\_main.c 做完配置解析之后，会调用：

```
skynet_start(&config);
```

这才是真正进入 Skynet Runtime 初始化。

把 skynet\_start() 主流程压缩一下：

```
初始化 Harbor / Handle
  ↓
初始化 Global Message Queue
  ↓
初始化 C Service Module Loader
  ↓
初始化 Timer
  ↓
初始化 Socket
  ↓
初始化 Profiling
  ↓
创建 logger Service
  ↓
创建 bootstrap Service
  ↓
创建 Monitor / Timer / Socket / Worker Threads
```

## 7.1 为什么 logger 和 bootstrap 在 Worker Thread 前创建

源码里先：

```
skynet_context_new(logger)
bootstrap(...)
```

最后才：

```
start(config->thread)
```

第一次看会觉得奇怪：线程都没启动，Service 怎么运行？

关键在于：**创建 Service Context 和真正调度其消息不是同一回事。**

Service Context、Handle、Mailbox 可以先创建好，初始化消息进入全局可运行队列；等 Worker Thread 建立后，再由 Worker 真正取出 Mailbox 开始执行。

这正好能帮助你建立 Skynet 的核心观念：

```
Service ≠ Thread
```

Service 是被 Runtime 调度的执行实体，不是“一创建 Service 就创建一个 pthread”。

## 8. thread = 8 到底创建了什么

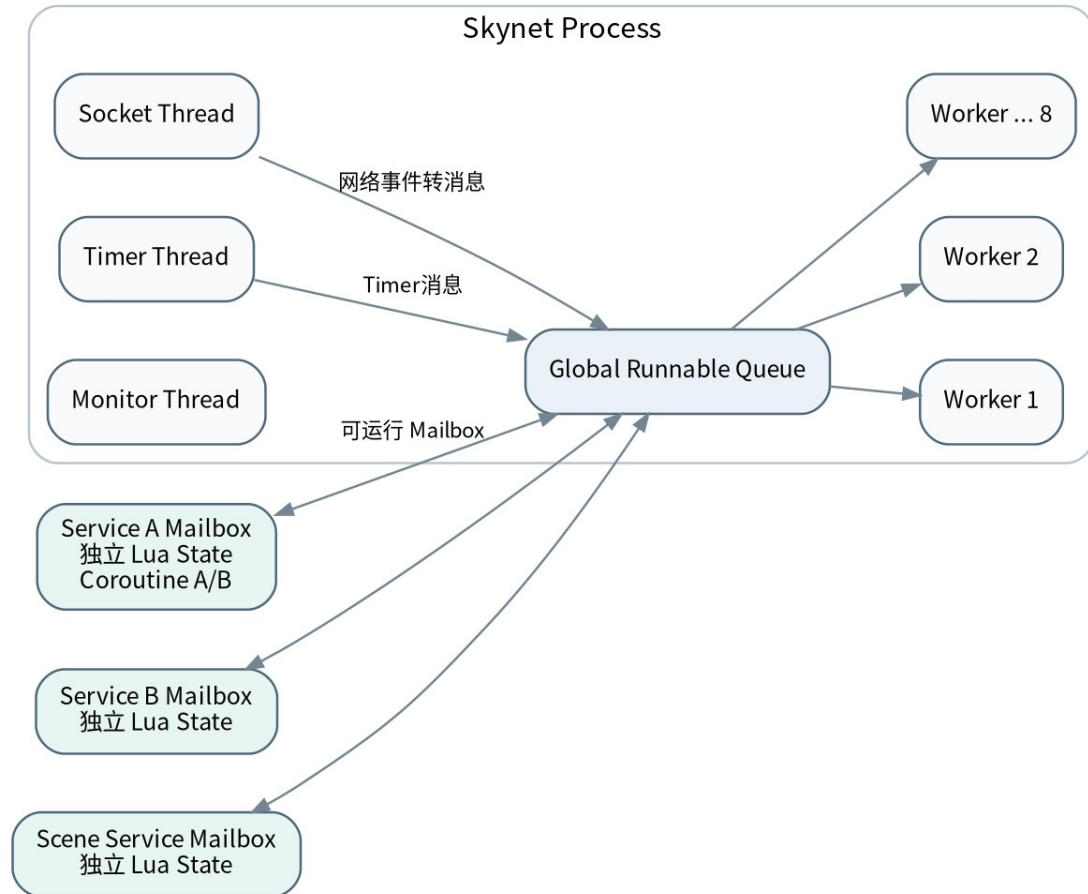
这是从传统多线程 GameServer 转 Skynet 时最值得彻底搞清楚的地方。

在 v1.8.0 的 skynet\_start.c 中，start(8) 会创建：

- 1 个 Monitor Thread；

- 1 个 Timer Thread;
- 1 个 Socket Thread;
- 8 个 Worker Thread。

原来的 Main Thread 则等待这些线程退出并做最终清理。



## 8.1 Worker 不固定绑定某个 Service

不要理解成：

```
Worker 1 = Scene1
Worker 2 = Scene2
Worker 3 = PlayerMgr
```

更接近：

```
很多 Service Mailbox
↓
Global Runnable Queue
↓
8 个 Worker 谁空闲谁取
```

同一个 Service 的不同消息，前后可以由不同 Worker 执行。

这与你熟悉的“某个 GameServer 固定一个主线程”很不一样。

## 8.2 thread = 8 不会把一个 Hot Scene 自动变成 8 核

假设一个大型场景只有一个：

```
SceneService
```

所有：

```
AOI
Move
Monster
Combat
```

都集中在这个 Service 里。

即使：

```
thread = 32
```

也不会把这个 Scene 的一条业务 Handler 自动拆成 32 线程并行。

Skynet 的多核利用来自：

```
多个可以同时运行的 Service
```

而不是：

```
一个 Service 内部被 Worker 自动切片并行
```

所以后面做 MMO 场景扩展时，如果单 Scene 成为热点，要考虑：

```
Scene 分线
Scene 分片
实例化地图
热点逻辑拆 Actor
```

而不是第一反应把 thread 从 8 改成 64。

## 8.3 一个 Service 同一时刻不会被两个 Worker 同时跑，但仍然要考虑“重入”

这一句需要非常精确：

```
同一个 Service 的 Lua State
不会同时在两个 Worker Thread 上执行 Lua 指令。
```

这帮你避免了大量传统共享内存多线程锁问题。

但它不等于：

```
一个 Handler 一旦开始，就一定完整执行到 return 后才处理下一件事。
```

因为 Skynet Lua 业务大量使用 Coroutine。

例如：

```
local gold = player.gold

local price = skynet.call(shop, "lua", "get_price", item_id)

player.gold = gold - price
```

skynet.call() 等待远端 Response 时，当前 Coroutine 会 yield。这个 Service 的 Lua State 没有被另一个 Worker 同时执行，但 Runtime 可以让同一个 Service 里的其他 Coroutine 继续工作。

于是可能出现：

```
Coroutine A: 读取 gold = 1000
Coroutine A: skynet.call -> yield

Coroutine B: 修改 gold = 1500

Coroutine A: 恢复
Coroutine A: 仍按旧的 1000 计算
```

这就是后面为什么会用：

```
skynet.queue()
```

来保护“虽然不会线程并行，但不能允许 Coroutine 交错”的关键业务区。

对于你来说，可以把 Skynet 的风险模型理解成：

```
少了很多 mutex / data race
但多了一类 await/yield 之后状态已经变化的时序问题
```

这比简单说“Actor 是单线程，所以不用考虑并发”准确得多。

---

## 9. bootstrap = "snlua bootstrap" 到底是什么

现在回到配置：

```
bootstrap = "snlua bootstrap"
```

这是第一课最重要的一行配置之一。

C Runtime 在 skynet\_start.c 中把这段字符串拆成：

```
C Service Name = snlua
Argument       = bootstrap
```

然后等价地走向：

```
skynet_context_new("snlua", "bootstrap");
```

所以千万不要把：

```
snlua bootstrap
```

理解成一个 Lua 文件名。

真正含义是：

```
创建一个 snlua 类型的 C Service，并把字符串 bootstrap 作为参数传进去。
```

### 9.1 snlua 是什么

上游文件：service-src/service\_snlua.c

snlua 可以粗略理解为：

**Skynet 提供的一种 C Service 外壳，它内部持有一个 lua\_State\*，把这个 Lua State 变成一个可被 Skynet Runtime 调度和收发消息的 Service。**

因此：

创建一次 snlua Service  
↓  
创建一个独立 Lua State

再创建另一个：

PlayerAgent

又有自己的 Lua State。

这就是为什么之前一直强调：

Skynet 不是全服共用一个 Lua VM。

## 9.2 snlua 初始化时给 Lua State 做了什么

不用背 C 代码，先理解它做的事：

创建 lua\_State  
↓  
打开 Lua 标准库  
↓  
安装 Skynet Coroutine/Profile 支持  
↓  
把 skynet\_context 指针放进 Lua Registry  
↓  
设置 LUA\_PATH / LUA\_CPATH / LUA\_SERVICE / LUA\_PRELOAD  
↓  
加载 loader.lua 指定的 loader.lua  
↓  
把 "bootstrap" 作为参数交给 loader.lua

这一层解释了前面 config/game.lua 里的：

```
luaservice
lua_loader
lua_path
lua_cpath
```

为什么存在。

它们不是一堆随便起的路径变量，而是在 snlua 建立每个 Lua Service 运行环境时被真正使用的。

## 10. loader.lua：从字符串 bootstrap 找到真正 Lua 文件

上游文件：third\_party/skynet/lualib/loader.lua

现在 snlua 已经有独立 Lua State，并把：

bootstrap

交给 loader.lua。

loader.lua 的核心逻辑可以概括为：

```

SERVICE_NAME = "bootstrap"

遍历 LUA_SERVICE 中的每个 Pattern
↓
把 ? 替换成 bootstrap
↓
loadfile()
↓
找到第一个存在的 Service 文件
↓
设置 package.path / package.cpath
↓
执行 Service 文件

```

本工程的 luaservice 是：

```

luaservice =
    "./service/? .lua;" ..
    "./third_party/skynet/service/? .lua"

```

所以它先试：

```
./service/bootstrap.lua
```

本工程没有这个文件。

然后再试：

```
./third_party/skynet/service/bootstrap.lua
```

命中。

于是终于进入官方：

```
service/bootstrap.lua
```

这一段流程非常值得亲自单步一次，因为以后你自己写：

```
skynet.newservice("player/player_agent")
```

本质还是同一套 Loader 规则。

## 11. 官方 bootstrap.lua：建立 Skynet 自己的系统 Service

上游文件：third\_party/skynet/service/bootstrap.lua

到了这里才真正开始执行第一个 Lua Service 的业务代码。

官方 bootstrap.lua 做的事情可以简化为：

```

启动 .launcher
↓
启动 .cslave / cdummy
↓
单节点模式下启动 DATACENTER

```

```

↓
启动 service_mgr
↓
读取 skynet.getenv("start")
↓
创建 main Service
↓
bootstrap 自己退出

```

当前工程：

```

harbor = 0
start = "main"

```

因此最后：

```
skynet.newservice("main")
```

会去寻找：

```
./service/main.lua
```

现在你应该能看出：

```
bootstrap = "snlua bootstrap"
```

和：

```
start = "main"
```

完全不是同一件事。

前者由 C Runtime 使用：

```

C Runtime
-> snlua bootstrap

```

后者由 Lua bootstrap Service 使用：

```

bootstrap.lua
-> skynet.getenv("start")
-> newservice("main")

```

如果以后有人说：“start = main 就是 Skynet Runtime 的启动入口。”

严格来说不准确。

更准确是：

bootstrap 是 Runtime 直接创建的 Bootstrap Service；start 是这个 Bootstrap Service 后续要启动的应用 Service 名称。

## 12. skynet.newservice() 不是 new Scene()

到这里需要停一下，专门把 newservice 的语义搞清楚。

你在业务里会经常写：

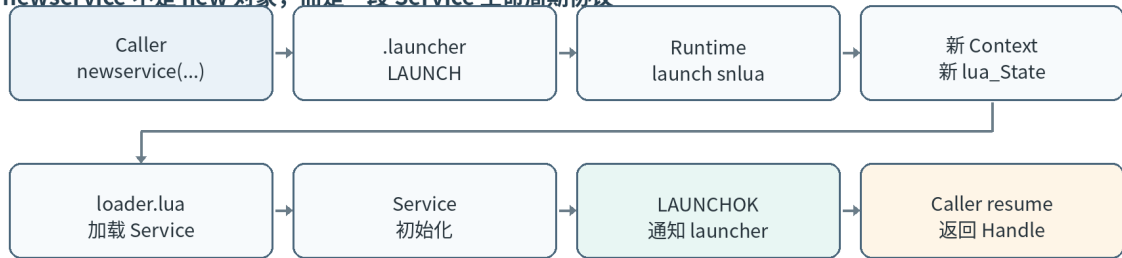
```
local scene = skynet.newservice("scene/scene", 1)
```

如果带着 C++ 习惯，很容易下意识理解成：

```
auto scene = new Scene(1);
```

但实际差别非常大。

**newservice 不是 new 对象，而是一段 Service 生命周期协议**



粗略流程是：

```

Caller
↓
skynet.newservice("scene/scene", 1)
↓
向 .launcher 发 LAUNCH 请求
↓
Caller Coroutine 等待 / yield
↓
.launcher 调 Runtime 创建 snlua Context
↓
新 Lua State
↓
loader.lua
↓
service/scene/scene.lua
↓
初始化成功
↓
新 Service 向 .launcher 报 LAUNCHOK
↓
.launcher 回复 Caller
↓
newservice 返回 Service Handle
  
```

这里有两个值得你特别注意的点。

## 12.1 newservice 是一次跨 Service 的生命周期协议

它不是普通对象构造。

所以它可能：

```

等待
失败
触发 Coroutine yield
  
```

这意味着如果你在某个状态敏感 Handler 中调用 newservice()，也要把它当成一个潜在的 yield 点审查。

## 12.2 返回的是 Handle，不是对象指针

```
local scene = skynet.newservice(...)
```

这里的 scene 是 Service Address/Handle。

后面你是：

```
skynet.call(scene, "lua", "enter", player)
```

而不是：

```
scene:Enter(player)
```

这正是 Actor / Message Passing 与共享内存对象模型的根本差异。

---

## 13. 终于来到我们的 service/main.lua

文件： service/main.lua

现在启动链已经走到：

```
C main
-> Runtime
-> snlua bootstrap
-> loader.lua
-> bootstrap.lua
-> newservice("main")
-> service/main.lua
```

这个 main.lua 是**我们项目自己的应用启动 Service**。

它不是整个 Linux 进程的 main，也不是 Skynet Runtime 的 main。

它更像你以前自研框架里的：

```
GameApplication::Init()
```

负责把业务依赖按顺序创建起来。

### 13.1 先看当前真实代码的启动顺序

当前仓库是：

```
ProtocolLoader
↓
ConfigService
↓
StorageMgr + Worker Pool
↓
SceneMgr + Default Scene
↓
PlayerMgr
↓
Auth
↓
Watchdog + Gate
```

```

↓
Debug Console
↓
main Service exit

```

这个顺序是有业务意义的，不是为了排版好看。

## 13.2 第一段：Protocol 和 Config

```

skynet.uniqueservice("protocol/protoloader")
skynet.uniqueservice("config/config_service")

```

uniqueservice 的意思可以先理解为：

当前 Skynet Node 内，按这个 Service Name 做唯一化创建。

它不是 C++ 语言层面的 static Singleton。

它解决的是：

“同一个节点里我只希望有一个 protocol loader / config service”

而不是：

“全世界所有服务器只能有一个”

将来你有多个 Game Node，每个 Node 都可以各自存在自己的 ConfigService。

## 13.3 StorageMgr 为什么先于 Scene / Player 启动

```

local storage_mgr = skynet.uniqueservice("storage/storage_mgr")

skynet.call(storage_mgr, "lua", "start", {
    driver = getenv("storage_driver", "memory"),
    pool = getenv_int("storage_pool", 4),
    ...
})

```

这里不是只“创建一个 Lua 文件”，而是：

1. 创建/取得 StorageMgr Service；
2. 再同步 call 它的 start 命令；
3. 当前 main Coroutine 等 StorageMgr 初始化完成。

为什么要等？

因为后面的 Player 登录需要 Storage。要是先开放 Gate，客户端已经进来了，而 DB/Memory Storage 还没 Ready，就会形成启动竞态。

这也是本工程把启动过程写成明确依赖链，而不是一股脑 send 出去的原因。

## 13.4 SceneMgr

```

local scene_mgr = skynet.uniqueservice("scene/scene_mgr")
skynet.call(scene_mgr, "lua", "start")

```

SceneMgr 的 start 返回后，业务才继续往下走。

这里也再次说明：

```
main.lua 是 Orchestrator
```

它并不自己持有地图实体、玩家或 AOI 数据。

## 13.5 PlayerMgr 通过“依赖注入”拿到 Storage 和 Scene

```
local player_mgr = skynet.uniqueservice("player/player_mgr")

skynet.call(player_mgr, "lua", "init", {
    storage_mgr = storage_mgr,
    scene_mgr = scene_mgr,
})
```

这段很值得保留。

PlayerMgr 不需要到处：

```
skynet.queryservice("storage_mgr")
```

或者硬编码 Service 名字。

而是由启动层把它真正依赖的 Handle 传进去。

从可维护性看，这与 C++ 中构造函数/初始化阶段显式传依赖是同一思想。

## 13.6 Gate 为什么最后才开放

```
local auth_service = skynet.uniqueservice("auth/auth")
local watchdog = skynet.uniqueservice("gateway/watchdog")

local address, port = skynet.call(watchdog, "lua", "start", {
    ...
    player_mgr = player_mgr,
    auth_service = auth_service,
})
```

前面已经依次准备好：

```
协议
配置
Storage
Scene
PlayerMgr
Auth
```

最后才让 Watchdog/Gate 开始 Listen。

这和传统服务器里常见的：

```
Init subsystem
Init DB
Load config
Init world
最后 Listen
```

是一样的工程原则。

如果 Gate 一开始就开放，内部依赖还在初始化，那么“进程已经起来”和“服务器可以接业务”就是两回事。

## 13.7 为什么 main.lua 最后可以 skynet.exit()

```
skynet.exit()
```

第一次看到很容易误解成：

```
服务器退出
```

实际上这里只是：

```
main 这个 Service 退出
```

已经创建的：

```
StorageMgr
SceneMgr
PlayerMgr
Auth
Watchdog
Gate
Debug Console
```

都有自己的 Service Context / Mailbox / Lua State，它们继续存活。

所以 main 最合适的职责就是：

```
启动时负责组织依赖；完成后退出，不再长期占一个业务角色。
```

这也是为什么它不应该顺手再承担什么 PlayerManager、WorldManager 状态。

---

## 14. 把四个最容易混的概念一次讲清楚

### 14.1 Service

你可以先把它理解成：

```
一个 Actor
+
一个 Mailbox
+
一个 Service Handle
+
自己的状态
```

如果是 snlua 类型 Service，还会有自己的：

```
lua_State
```

### 14.2 Worker Thread

Worker 是 Runtime 的执行资源。

它的工作不是“拥有某个 Service”，而是不断从可运行队列取 Service Mailbox 并处理消息。

所以关系是：

```
Service : Worker = M : N
```

而不是 1:1。

## 14.3 Lua State

每个 snlua Service 有自己的 Lua State。

因此：

```
local players = {}
```

定义在 `player_mgr.lua` 中，是 PlayerMgr 自己 Lua State 里的 table。

另一个 Scene Service 不会天然拿到这张 table 的引用。

跨 Service 通信应通过：

```
skynet.call  
skynet.send
```

而不是共享普通 Lua Object。

## 14.4 Coroutine

一个 Lua Service 内部又可以存在多个 Coroutine。

例如：

```
Coroutine A: 处理玩家 move  
Coroutine B: 等待 storage call  
Coroutine C: Timer callback
```

它们仍然属于同一个 Lua State，所以会共享 Service 内的：

```
local player  
local state  
local cache
```

这就是为什么：

```
没有多线程 data race
```

并不等于：

```
不需要考虑并发时序
```

Skynet 业务 Code Review 里，一个非常重要的问题就是：

**这段函数中间有哪些地方可能 yield? yield 回来以后，我之前读到的状态还可信吗？**

对于有多年单线程 MMO GameServer 经验的人，这个思维切换比记 API 更重要。

## 15. 顶层 Lua Chunk 和 skynet.start() 也不是同一个阶段

这一点属于进阶，但第一课最好先建立概念。

一个 Service 文件例如：

```
service/scene/scene.lua
```

被 loader.lua 加载时，首先执行的是 Lua 文件顶层代码：

```
local skynet = require "skynet"
local CMD = {}
local entities = {}

local function foo()
    ...
end

skynet.start(function()
    ...
end)
```

你可以把它分成：

### 阶段 A：加载文件顶层 Chunk

做：

```
require
创建 local table
定义函数
注册 skynet.start(...)
```

### 阶段 B：Service 初始化 Callback

再由 Skynet 调度执行：

```
skynet.start(function()
    ...真正初始化...
end)
```

为什么这一区分有用？

因为以后遇到：

```
newservice failed
```

你要继续判断：

```
是 loader 根本没找到文件？
是顶层 require 报错？
还是 skynet.start() 里面初始化依赖时报错？
```

错误层次不同，调试方式也不同。

## 16. 从 C++ MMO Server 视角重新映射一次

下面这个表不是严格等价，只用来快速建立直觉。

| Skynet           | 传统 C++ MMO Server 中接近的概念  | 不能忽略的差异                            |
|------------------|---------------------------|------------------------------------|
| skynet 主程序       | Server Host / Runtime     | 自带 Actor 调度、Mailbox、Timer、Socket 等 |
| Service Handle   | Actor ID / Runtime Handle | 不是对象指针，跨 Service 用消息               |
| Service Mailbox  | Actor 消息队列                | Worker 动态调度，不固定线程                  |
| snlua Service    | 带独立 Lua VM 的 Actor        | 每个实例自己的 Lua State                  |
| skynet.call      | Async RPC + await         | 当前 Coroutine 会 yield               |
| skynet.send      | Fire-and-forget Message   | 不等待结果                              |
| uniqueservice    | 节点内命名的唯一 Actor            | 不是跨集群 Singleton                    |
| Worker Thread    | Job System Worker         | 不固定绑定某个业务对象                        |
| service/main.lua | Application Bootstrap     | 只负责启动依赖，完成后可退出                     |

如果你以前的 GameServer 模型是：

```
一个 Game Thread
↓
PlayerManager
SceneManager
GuildManager
ActivityManager
```

Skynet 更接近：

```
很多状态所有权明确的 Service
↓
Runtime 用多个 Worker 调度
↓
Service 之间通过 Message 连接
```

最核心的问题也从：

“这个对象由哪个 Manager 持有？”

逐渐变成：

“这份状态的权威所有者是哪个 Service？”

## 17. 亲手验证，比继续读十页源码更有用

理解启动链以后，建议马上做几个小实验。

## 17.1 实验一：确认构建产物

在仓库根目录：

```
file third_party/skynet/skynet
file third_party/skynet/cservice/snlua.so
file third_party/skynet/luac/lib/skynet.so
```

你应该能明确回答：

- skynet：主程序；
- snlua.so：C Service；
- skynet.so：Lua C Module。

接着：

```
ldd third_party/skynet/skynet
```

看看主程序的动态依赖。

这一步的目标不是背 ldd 输出，而是把“主程序 / C Service / Lua C Module”三层真正和磁盘文件对应起来。

## 17.2 实验二：GDB 从 main 跟到 Runtime

```
gdb --args ./third_party/skynet/skynet config/game.lua
```

先只下三个断点：

```
break main
break skynet_start
break skynet_context_new
run
```

**在 main 看**

```
print argc
print argv[1]
```

确认：

```
argv[1] = config/game.lua
```

**在 skynet\_start 看**

```
print config->thread
print config->bootstrap
print config->module_path
```

你会直接看到 config/game.lua 经过临时 Lua State 解析后，已经变成 C Runtime 使用的配置。

**在 skynet\_context\_new 看**

反复：

```
print name
print param
continue
```

开始时重点观察：

```
logger
snlua / bootstrap
```

之后才会逐步出现 launcher、main 和业务 Service。

第一次建议不要在所有 Message Dispatch 上打断点，会命中到几乎没法看。

### 17.3 实验三：给 snlua\_init 下动态库断点

因为 snlua\_init 在运行时加载的 snlua.so 中：

```
set breakpoint pending on
break snlua_init
run
```

命中后：

```
bt
```

这一次你看的不是某个 Lua 业务，而是：

```
C Runtime 怎样建立一个 Lua Service
```

这一步对理解 snlua 很有价值。

### 17.4 实验四：故意改坏 luaservice

在单独分支临时把：

```
luaservice = "./not_exists/?lua"
```

然后启动。

你会看到：

- C Runtime 已经起来；
- snlua 也可以被加载；
- 但 loader.lua 找不到 bootstrap.lua。

这就叫：

```
Lua Service Loader 阶段失败
```

而不是 Runtime Module Loader 失败。

实验结束后恢复配置。

### 17.5 实验五：故意让 service/main.lua 的初始化报错

例如在测试分支的：

```
skynet.start(function()
```

开头临时：

```
error("lesson1 test error")
```

这次和上一个实验不同：

```
loader 已经找到 main.lua
顶层 Chunk 也已经进入
但业务 Service 初始化失败
```

你要训练自己根据错误发生的层级判断问题，而不是看到“服务起不来”就全部归为同一种故障。

## 18. 一套实用的启动故障定位顺序

以后 Server 启动失败，可以按这个顺序判断，不需要一上来全局搜日志。

| 现象                                  | 优先怀疑层次                  | 首先看什么                               |
|-------------------------------------|-------------------------|-------------------------------------|
| config/game.lua 找不到 /<br>Lua 配置语法错误 | 配置阶段                    | stderr、skynet_main.c                |
| snlua / logger 无法加载                 | C Service Module        | cpath、.so、<br>skynet_module_query   |
| bootstrap.lua / main.lua 找不到        | Lua Service Loader      | luaservice、loader.lua               |
| require 某模块失败                       | Lua Module Loader       | lua_path / lua_cpath                |
| newservice 一直不返回                    | Service 初始化 / .launcher | Debug Console task、目标<br>Service 日志 |
| 进程存在但 8888 没监听                      | 业务启动链未完成                | service/main.lua、<br>Watchdog/Gate  |
| 单 Scene CPU 满，改 thread<br>效果很小      | Hot Actor               | Scene 内逻辑、Actor 粒度                  |

这种层次化排查是这节源码课真正能带到生产环境里的价值。

## 19. 第一课结束前，再把整条链自己读一遍

现在不要看前文，尝试只看下面这段并在脑子里补全每一步发生了什么：

```
./third_party/skynet/skynet config/game.lua
↓
skynet_main.c::main
↓
临时 Lua State 执行 config/game.lua
↓
配置写入 skynet_env
↓
lua_close(config_state)
```

↓

skynet\_start(&config)

↓

初始化 Handle / MQ / Module / Timer / Socket

↓

logger

↓

bootstrap = "snlua bootstrap"

↓

skynet\_context\_new("snlua", "bootstrap")

↓

snlua 创建独立 lua\_State

↓

loader.lua

↓

third\_party/skynet/service/bootstrap.lua

↓

.launcher / .cslave / DATACENTER / service\_mgr

↓

skynet.getenv("start") == "main"

↓

skynet.newservice("main")

↓

service/main.lua

↓

Protocol / Config / Storage / Scene / Player / Auth

↓

Watchdog + Gate 最后开放

↓

main Service exit

↓

MMO Server 继续运行

如果上面任何一行你还说不清“谁调用谁、处在哪个 Lua State、会不会新建 Service、会不会 yield”，就回到对应章节再看一次。

## 20. 本课你真正需要记住的 8 件事

1. `config/game.lua` 是由 `skynet_main.c` 创建的**临时 Lua State** 执行，执行完就销毁，它不是业务 Lua VM。
2. 配置最终进入 Skynet Runtime Environment，业务 Service 通过 `skynet.getenv()` 读取。
3. `thread = 8` 表示 **8 个 Worker Thread**，另外还有 Monitor、Timer、Socket Thread；它不等于一个 Service 可以 8 核并行执行。
4. `bootstrap = "snlua bootstrap"` 的意思是：创建 **snlua C Service**，参数是 `bootstrap`。
5. 一个 `snlua` Service 拥有自己的 `lua_State`；Skynet 不是整个进程共享一个 Lua State。
6. `loader.lua` 根据 `luaservice` 找真正的 Lua Service 文件；`require()` 则主要走 `lua_path/lua_cpath`，二者不要混。
7. `start = "main"` 不是 C Runtime 直接消费的应用入口，而是官方 `bootstrap.lua` 后续读取并创建的业务 Service。
8. “一个 Service 不会被两个 Worker 同时执行”不代表“业务永不重入”。`skynet.call/sleep/wait` 造成的 Coroutine yield 仍然要求你做时序审查。

## 21. 自测题：不要背答案，最好自己画

### 题 1

为什么 `config/game.lua` 中不能直接创建一个 Lua table，然后让所有 `PlayerAgent` 共享这个 table？

**\*\*回答要点：**\*\*配置文件由临时 Lua State 执行，之后被销毁；不同 `snlua` Service 又有各自独立 Lua State。共享静态配置要使用专门机制，例如本工程后面使用的 `sharedata`。

### 题 2

`bootstrap = "snlua bootstrap"` 中，前一个 `snlua` 和后一个 `bootstrap` 分别是什么？

**\*\*回答要点：**\*\*前者是 C Service Module；后者是传给该 C Service 的参数，最终被 `loader` 当作 Lua Service Name 找到官方 `bootstrap.lua`。

### 题 3

为什么 `thread = 32` 不一定能解决某个 Scene Service CPU 100%?

**回答要点：**Worker 提升的是多个 Service 的并行度；单个 Hot Actor 仍然是单 Service 状态所有者，需要从 Scene 分线、分片、实例化等 Actor 粒度入手。

### 题 4

`skynet.newservice("scene/scene")` 和 C++ `new Scene()` 最大的三个差别是什么？

**回答要点：**它会创建一个 Runtime Service/可能创建独立 Lua State；通过 `.launcher` 完成异步生命周期握手；返回的是 Service Handle，后续通过消息通信而不是对象方法调用。

### 题 5

为什么 `service/main.lua` 可以最后 `skynet.exit()`，而整个 MMO Server 不会退出？

**回答要点：**`skynet.exit()` 退出的是当前 `main Service`。其他已创建 Service 有独立 Context/Handle/Mailbox，仍然继续运行。

### 题 6

为什么一个 Service 没有传统多线程 Data Race，仍然可能出现类似“旧值覆盖新值”的业务 Bug？

**回答要点：**Handler 在 `skynet.call` 等位置 `yield` 后，其他 Coroutine 可以修改同一 Lua State 中的共享状态；原 Coroutine 恢复时之前读取的状态可能已经过期。

## 22. 下一课应该怎么接

第一课把“进程怎么起来、Service 怎么诞生”讲清楚以后，下一课最自然的切入点不是继续看更多启动源码，而是追踪一条真正的业务消息。

建议下一课直接从：

```
Client 发 move
```

开始，一路追：

```

TCP Socket Event
-> Gate
-> forward / redirect
-> PlayerAgent Mailbox
-> PTYPE_CLIENT
-> Sproto dispatch
-> PlayerAgent REQUEST.move
-> skynet.call(Scene)
-> Caller Coroutine yield
-> Scene Service 处理
-> Response
-> PlayerAgent Coroutine resume
-> Server Push / Client Response

```

到了那一课，第一课里的：

```
Worker
Mailbox
Service Handle
Lua State
Coroutine
call/yield
```

会从“启动概念”变成真正的运行时行为。

---

## 附录 A：本课源码阅读索引

### 本工程

```
Makefile
scripts/bootstrap_skynet.sh
scripts/linux/build.sh
scripts/linux/run_server.sh
config/game.lua
service/main.lua
```

仓库：

```
https://github.com/simbiwu/skynet-mmo-learning
```

本文对应仓库基线：

```
b6a55b59369565dbe7fd82237f1d1020aaff543c
```

### Skynet v1.8.0 上游

```
skynet-src/skynet_main.c
skynet-src/skynet_start.c
service-src/service_snlua.c
lua-lib/loader.lua
service/bootstrap.lua
service/launcher.lua
```

上游：

```
https://github.com/cloudwu/skynet/tree/v1.8.0
```

---

## 附录 B：建议你实际执行的一组命令

### Bootstrap + Build

```
make bootstrap
make build
```

## 启动服务器

```
make server
```

等价核心命令：

```
./third_party/skynet/skynet config/game.lua
```

## 查看产物类型

```
file third_party/skynet/skynet
file third_party/skynet/cservice/snlua.so
file third_party/skynet/luaclib/skynet.so
```

## 查看动态依赖

```
ldd third_party/skynet/skynet
```

## GDB

```
gdb --args ./third_party/skynet/skynet config/game.lua
break main
break skynet_start
break skynet_context_new
run
```

动态模块：

```
set breakpoint pending on
break snlua_init
run
```